

Random Forest Models for Tree Species Habitat Suitability — Abiotic and Co-occurrence Predictors

João Paulo Czarnecki de Liz

2026-06-26

Contents

Overview	2
0. Spatial Blocking — Train/Test Partitioning	4
1. Data Loading	5
2. Community-Level IVI and Species Prioritization (Pareto Analysis)	5
3. Plot-Level IV and Leave-One-Out Co-occurrence Normalization	7
3.1 calc_VIS_LOO — Leave-One-Out Function Definition	7
3.2 LOO VIS for Training Set	8
4. Spatial Blocking — Block Size Determination	9
5. Variable Selection — Altmann Permutation Method (Step 1)	10
6. Final Abiotic Models — 1,000 Trees (Step 2)	15
7. Test Set LOO VIS and Independent Test Set Construction	17
8. Final Abiotic + Co-occurrence Models — 1,000 Trees (Step 3)	19
9. Model Consolidation	21
10. Performance Evaluation — Training and Independent Test Set	21
10.1 Prediction Helpers	21
10.2 Metrics Calculation	24
11. Observed vs. Predicted Plots	24
12. Spatial Maps — Hexagon Maps	26

Overview

This script implements the full modeling pipeline described in:

Czarnecki de Liz, J.P. et al. (2026). *Integrating Phytosociological Proxies to Improve Predictions of Tree Habitat Suitability: Potentials and Limitations*.

Response variable: Plot-level Importance Value (IV) for seven prioritized boreal-temperate tree species. **Predictors:** 40 abiotic candidates (topography, climate, soil) $\pm$ leave-one-out co-occurrence IV of six co-occurring species. **Engine:** Random Forest via `ranger` + `caret`, with spatial cross-validation (`blockCV`).

```
suppressPackageStartupMessages({
  library(tidyverse)
  library(caret)
  library(ranger)
  library(blockCV)
  library(scales)
  library(sf)
  library(doParallel)
  library(purrr)
  library(ggpubr)
  library(ggQC)
  library(patchwork)
  library(viridis)
  library(glue)
  library(ggspatial)
  library(rnaturalearth)
  library(readr)
})

pak::pak("kenithgrey/ggQC")
install.packages("scales")
# =====
# CONFIGURATION - edit this block to adapt the pipeline
# =====
set.seed(1304)

# Seven prioritized species (80% cumulative IVI threshold; see Section 2)
species_list <- c("SAB", "BOP", "BOJ", "EPN", "EPB", "ERR", "ERS")

# Species name mapping: code → full common name (used in plots and exports)
qc_species_names <- c(
  "SAB_n" = "Balsam_fir", "BOP_n" = "Paper_birch", "BOJ_n" = "Yellow_birch",
  "EPN_n" = "Black_spruce", "EPB_n" = "White_spruce", "ERR_n" = "Red_maple",
  "ERS_n" = "Sugar_maple"
```

```

)

# species_map: code + full name with spaces (for axis labels)
species_map <- setNames(
  gsub("_", " ", qc_species_names),
  sub("_n$", "", names(qc_species_names))
)

# 40 abiotic candidate predictors
abiotic_candidates <- c(
  # Topography
  "Slope", "TPI", "TRI", "TWI", "Curv_class",
  "Insol_direct", "Insol_diffuse", "Wind_exposition", "AspectClass",
  # Soil
  "Clay", "CEC", "Organic_Matter", "pH", "Silt", "Sand",
  # Climate (ClimateNA normals 1991-2020)
  "MAT", "MWT", "MCMT", "TD", "MAP", "MSP", "AHM", "SHM",
  "DD_0", "DD5", "DD_18", "DD18", "NFFD", "bFFP", "eFFP", "FFP",
  "PAS", "EMT", "EXT", "MAR", "Eref", "CMD", "RH", "CMI", "DD1040"
)

# Factor variables among abiotic candidates
factor_vars <- c("Curv_class", "AspectClass")

# Spatial blocking - block_size from spatial autocorrelation
# (see Section 4)
block_size <- 8658 # metres
k_folds <- 5

# Variable selection (Step 1 - selection models)
n_trees_selection <- 800
alpha_pvalue <- 0.05 # Altmann permutation significance threshold
min_keep <- 6 # minimum predictors retained per species
altmann_permutations <- 30

# Final models (Steps 2 & 3)
n_trees_final <- 1000

# Pre-processing pipeline applied by caret
pre_proc <- c("nzv", "corr", "YeoJohnson", "center", "scale")

# Hyperparameter search grid
mtry_candidates <- c(2, 5, 8, 10, 15, 20)

# Output directory (timestamped)
out_dir <- file.path(
  "results",
  paste0("RF_models_", format(Sys.time(), "%Y%m%d_%H%M%S"))
)
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(out_dir, "maps"), recursive = TRUE, showWarnings = FALSE)
dir.create(
  file.path(out_dir, "results"),
  recursive = TRUE, showWarnings = FALSE
)

```

```
)
```

0. Spatial Blocking — Train/Test Partitioning

Run once. This chunk generates the train/test CSV files consumed by Section 1. Skip (set `eval = FALSE`) after the files have been saved.

The dataset is split into one training fold and one test fold using a single spatial block cross-validation (`blockCV::cv_spatial`, `k = 2`). Block size was derived from the range of spatial autocorrelation (see Section 4).

```
plots_pet <- read_csv(
  "database/undisturbed_plots_sf_env_clim_4th_inv_study_area_PET.csv",
  show_col_types = FALSE
) |>
  mutate(across(any_of(factor_vars), as.factor))

plots_pet_sf <- st_as_sf(
  plots_pet, coords = c("longitude", "latitude"), crs = 2950
)

sb1 <- cv_spatial(
  x          = plots_pet_sf,
  size       = block_size,
  k          = 2,
  hexagon    = FALSE,
  selection   = "random",
  seed       = 1304,
  report     = FALSE,
  progress   = FALSE
)

train_indices <- sb1$folds_list[[1]][[1]]
test_indices  <- sb1$folds_list[[1]][[2]]

trainData <- plots_pet[train_indices, ]
testData  <- plots_pet[test_indices, ]

write_csv(trainData, "database/trainData_vis_IVI_1991_2020Y_27feb26.csv")
write_csv(testData, "database/testData_vis_IVI_1991_2020Y_27feb26.csv")

message(
  "Saved - Training: n = ", nrow(trainData),
  " | Test: n = ", nrow(testData)
)
```

1. Data Loading

Training and test sets were partitioned by spatial blocking (Section 0). Files include plot-level IV for each species, abiotic predictors, and raw abundance/basal area columns.

```
trainData <- read_csv("database/trainData_vis_IVI_1991_2020Y_27feb26.csv",
  show_col_types = FALSE)
testData <- read_csv("database/testData_vis_IVI_1991_2020Y_27feb26.csv",
  show_col_types = FALSE)
study_area <- st_read("database/study_area_ok.shp", quiet = TRUE) %>%
  st_transform(crs = 2950)

# Detect coordinate columns
coord_cols <- if (all(c("longitude", "latitude") %in% names(trainData))) {
  c("longitude", "latitude")
} else if (all(c("x", "y") %in% names(trainData))) {
  c("x", "y")
} else {
  stop("Coordinate columns not found. Expected 'longitude/latitude' or 'x/y'.")
}

# Coerce factor predictors
trainData <- trainData %>% mutate(across(any_of(factor_vars), as.factor))
testData <- testData %>% mutate(across(any_of(factor_vars), as.factor))

# Resolve which abiotic candidates are actually present
abiotic_vars <- intersect(abiotic_candidates, names(trainData))

# Verify species response columns exist
missing_sp <- setdiff(species_list, names(trainData))
if (length(missing_sp) > 0) stop("Missing species columns in trainData: ",
  paste(missing_sp, collapse = ", "))

message("Training set: n = ", nrow(trainData),
  " | Test set: n = ", nrow(testData))
message("Abiotic predictors available: ", length(abiotic_vars),
  " of ", length(abiotic_candidates))
```

2. Community-Level IVI and Species Prioritization (Pareto Analysis)

IVI was computed at community level following Curtis & McIntosh (1951): $IVI = (\text{relative density} + \text{relative dominance} + \text{relative frequency}) / 3$

This community-level IVI is used **only** for species ranking. The response variable for modeling is the plot-level IV (see Section 3), which excludes relative frequency (not meaningful at the plot scale).

```

# Raw abundance and basal area matrices
n_comm <- trainData %>%
  select(ends_with("_n")) %>%
  rename_with(~ sub("_n$", "", .x))

st_comm <- trainData %>%
  select(ends_with("_ba")) %>%
  rename_with(~ sub("_ba$", "", .x))

# Community-level IVI components
da <- colSums(n_comm, na.rm = TRUE) / nrow(n_comm) # Absolute density
dr <- (da / sum(da)) * 100 # Relative density (%)
doa <- colSums(st_comm, na.rm = TRUE) / nrow(st_comm) # Absolute dominance
dor <- (doa / sum(doa)) * 100 # Relative dominance (%)
fa <- colSums(n_comm != 0, na.rm = TRUE) # Absolute frequency (n plots)
fr <- (fa / sum(fa)) * 100 # Relative frequency (%)
ivi <- (dr + dor + fr) / 3 # IVI

IVI_Curtis <- as.data.frame(cbind(da, dr, doa, dor, fa, fr, ivi)) %>%
  filter(ivi > 0) %>%
  arrange(desc(ivi)) %>%
  mutate(
    species = rownames(.),
    cum_perc = cumsum(ivi) / sum(ivi) * 100
  )

write_csv(IVI_Curtis, file.path(out_dir, "IVI_Curtis_community_level.csv"))

# Pareto chart - 80% cumulative IVI threshold
vline_x <- which.min(abs(IVI_Curtis$cum_perc - 80))

p_pareto <- ggplot(IVI_Curtis, aes(x = reorder(species, -ivi), y = ivi)) +
  stat_pareto(
    point.color = "black",
    point.size = 0.5,
    line.color = "black",
    bars.fill = c("blue", "green", "orange")
  ) +
  geom_vline(
    xintercept = vline_x,
    linetype = "dashed",
    color = "grey40",
    linewidth = 0.5
  ) +
  labs(
    title = "Species Prioritization by Importance Value Index (IVI)",
    x = "Species",
    y = "IVI"
  ) +
  theme_pubr() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

print(p_pareto)
ggsave(

```

```

file.path(out_dir, "Pareto_IVI_species_prioritization.png"),
plot = p_pareto, width = 9, height = 4, dpi = 300
)

```

3. Plot-Level IV and Leave-One-Out Co-occurrence Normalization

3.1 calc_VIS_L00 — Leave-One-Out Function Definition

```

# =====
# calc_VIS_L00()
#
# Computes plot-level co-occurrence Importance Value (IV = (RA + RD) / 2) using
# leave-one-out normalization. For each focal species, relative abundance
# (RA) and relative dominance (RD) of co-occurring species are computed by
# normalizing each row by the plot total EXCLUDING the focal species.
# This prevents the closed-sum constraint from creating structural
# dependence between predictors and response.
#
# Arguments:
#   n_mat      : data.frame of stem counts per species (one row per plot)
#   ba_mat      : data.frame of basal area per species (one row per plot)
#   target      : character code of the focal species (e.g. "SAB")
#   scale100     : logical; multiply proportions by 100 (default TRUE)
#   drop_target_col : logical; remove the focal species column from
#                   output (default TRUE)
#
# Returns: data.frame of L00-normalized co-occurrence IV (one row per plot)
# =====

calc_VIS_L00 <- function(n_mat, ba_mat, target,
                        scale100 = TRUE,
                        drop_target_col = TRUE) {
  # Standardize column names (strip _n / _ba suffixes if present)
  colnames(n_mat) <- gsub("_n$|_ba$", "", colnames(n_mat))
  colnames(ba_mat) <- gsub("_n$|_ba$", "", colnames(ba_mat))

  n_mat <- n_mat[, !duplicated(colnames(n_mat)), drop = FALSE]
  ba_mat <- ba_mat[, !duplicated(colnames(ba_mat)), drop = FALSE]

  n <- as.matrix(n_mat)
  ba <- as.matrix(ba_mat)

  stopifnot(!is.null(colnames(n)), !is.null(colnames(ba)))

  common_species <- intersect(colnames(n), colnames(ba))
  if (length(common_species) == 0) {
    stop("No common species names found between n_mat and ba_mat.")
  }
}

```

```

n <- n[, common_species, drop = FALSE]
ba <- ba[, common_species, drop = FALSE]

if (!(target %in% colnames(n))) {
  stop(paste("Target species", target, "not found in column names."))
}

# Leave-one-out denominators: row totals minus the focal species
denom_n <- rowSums(n) - n[, target]
denom_ba <- rowSums(ba) - ba[, target]

denom_n[denom_n <= 0] <- NA_real_
denom_ba[denom_ba <= 0] <- NA_real_

# Relative abundance and relative dominance (LOO-normalized)
ra <- sweep(n, 1, denom_n, "/")
rd <- sweep(ba, 1, denom_ba, "/")

if (scale100) {
  ra <- ra * 100
  rd <- rd * 100
}

vis <- (ra + rd) / 2

if (drop_target_col) {
  vis <- vis[, setdiff(colnames(vis), target), drop = FALSE]
}

vis[!is.finite(vis)] <- 0
return(as.data.frame(vis))
}

```

3.2 LOO VIS for Training Set

```

n_train <- trainData %>%
  select(ends_with("_n")) %>%
  rename_with(~ sub("_n$", "", .x))

st_train <- trainData %>%
  select(ends_with("_ba")) %>%
  rename_with(~ sub("_ba$", "", .x))

species_loo <- intersect(
  species_list,
  intersect(colnames(n_train), colnames(st_train))
)

if (length(species_loo) == 0)
  stop("No target species found with both _n and _ba columns.")

VIS_LOO_list <- setNames(vector("list", length(species_loo)), species_loo)

```

```

for (target in species_loo) {
  message("Computing training L00 VIS for: ", target)
  VIS_L00_list[[target]] <- calc_VIS_L00(
    n_mat = n_train,
    ba_mat = st_train,
    target = target,
    scale100 = TRUE,
    drop_target_col = TRUE
  )
}

saveRDS(VIS_L00_list, file.path(out_dir, "results/VIS_L00_list.rds"))
message("VIS_L00_list saved.")

```

4. Spatial Blocking — Block Size Determination

Block size (8,658 m) was determined by `blockCV::cv_spatial_autocor()` applied to a 100 m environmental raster. This chunk is `eval = FALSE` to skip re-running the computationally intensive step; the derived value is hardcoded in the configuration block above.

```

raster_env <- terra::rast(
  "database/UPE_1415NC/raster/env_clim_raster_100m.tif"
)

set.seed(1304)
sac1 <- cv_spatial_autocor(r = raster_env, num_sample = 20000)

dir.create(
  file.path(out_dir, "results/blockCV_results"),
  recursive = TRUE, showWarnings = FALSE
)
saveRDS(
  sac1$range,
  file.path(out_dir, "results/blockCV_results/sac1_range.rds")
)
write.csv(
  sac1$range_table,
  file.path(out_dir, "results/blockCV_results/sac1_range_table.csv"),
  row.names = FALSE
)

# sac1$range = 8658 + stored as block_size = 8658 in configuration

```

5. Variable Selection — Altmann Permutation Method (Step 1)

An initial Random Forest (800 trees) was trained per species using all 40 abiotic candidates. Variable importance was tested for statistical significance using the Altmann permutation method (30 permutations, $\alpha = 0.05$). A minimum of 6 predictors was always retained per species.

```
# Parallel backend
n_cores <- max(1, parallel::detectCores() - 1)
cl <- parallel::makePSOCKcluster(n_cores)
parallel::clusterEvalQ(cl, .libPaths(c("H:/R_lib", .libPaths())))
doParallel::registerDoParallel(cl)
on.exit(try(parallel::stopCluster(cl), silent = TRUE), add = TRUE)

selection_models <- list()
selected_vars_abio <- list()
selection_importance_tables <- list()
controls_by_species <- list()

for (sp in species_list) {
  message("[Variable Selection] ", sp)

  # Complete-case subset for this species
  needed <- unique(c(coord_cols, abiotic_vars, sp))
  df_sp <- trainData %>%
    select(any_of(needed)) %>%
    filter(complete.cases(., intersect(needed, names(.))))

  if (nrow(df_sp) < 100) {
    message("Skipping ", sp, " - too few complete rows (", nrow(df_sp), ")")
    next
  }

  # Spatial cross-validation folds
  sb <- tryCatch(
    {
      sf_df <- st_as_sf(df_sp, coords = coord_cols, crs = 2950, remove = FALSE)
      cv_spatial(
        x = sf_df, size = block_size, k = k_folds,
        selection = "random", iteration = 100, seed = 1304, progress = FALSE
      )
    },
    error = function(e) {
      message(
        "cv_spatial failed for ", sp,
        "; falling back to standard repeatedcv."
      )
      NULL
    }
  )

  ctrl_args <- list(
    method = "repeatedcv",
    number = k_folds,
    repeats = 3,
  )
}
```

```

    savePredictions = "final",
    allowParallel   = TRUE,
    verboseIter     = FALSE
  )
  if (!is.null(sb)) {
    ctrl_args$index <- lapply(sb$fold_list, `[`, 1)
    ctrl_args$indexOut <- lapply(sb$fold_list, `[`, 2)
  }
  control <- do.call(caret::trainControl, ctrl_args)
  controls_by_species[[sp]] <- control

  # Tune grid (cap mtry at number of available predictors)
  p <- length(abiotic_vars)
  tune_grid <- expand.grid(
    mtry      = sort(unique(pmax(1, pmin(p, mtry_candidates)))),
    splitrule = "variance",
    min.node.size = c(5, 10)
  )

  model_df <- df_sp %>%
    select(all_of(c(abiotic_vars, sp))) %>%
    rename(occ = all_of(sp)) %>%
    mutate(occ = as.numeric(occ)) %>%
    filter(complete.cases(.))

  set.seed(1304)
  model_sel <- caret::train(
    occ ~ .,
    data      = model_df,
    method    = "ranger",
    preProcess = pre_proc,
    metric     = "RMSE",
    trControl  = control,
    tuneGrid   = tune_grid,
    num.trees  = n_trees_selection,
    importance = "permutation"
  )

  rf <- model_sel$finalModel
  imp <- ranger::importance(rf)

  # Map dummy-coded factor columns back to original variable names
  orig_factor_map <- list()
  for (fv in factor_vars) {
    dummies <- grep(paste0("^", fv), names(imp), value = TRUE)
    for (dm in dummies) orig_factor_map[[dm]] <- fv
  }

  imp_tbl <- tibble(
    Variable = names(imp),
    Importance = as.numeric(imp),
    OriginalVar = vapply(
      names(imp),
      function(v) {

```

```

    if (v %in% names(orig_factor_map)) orig_factor_map[[v]] else v
  },
  character(1)
)
)

# Altmann permutation p-values (Pomona + ranger fallback + rank-only)
method_used <- "altmann"

p_tbl <- tryCatch(
{
  if (!requireNamespace("Pomona", quietly = TRUE))
    stop("Pomona not installed")

  tr <- rename(model_sel$trainingData, occ = .outcome)
  tr <- tr[complete.cases(tr), , drop = FALSE]
  xnames <- if (!is.null(rf$xNames)) {
    rf$xNames
  } else {
    rf$forest$independent.variable.names
  }
  if (is.null(xnames)) stop("Predictor names not found in finalModel")

  X <- if (!is.null(model_sel$terms)) {
    mm <- model.matrix(model_sel$terms, data = tr)
    as.data.frame(mm[, setdiff(colnames(mm), "(Intercept)"), drop = FALSE])
  } else {
    as.data.frame(tr[, setdiff(names(tr), "occ"), drop = FALSE])
  }

  missing_cols <- setdiff(xnames, names(X))
  if (length(missing_cols) > 0)
    stop("Predictor mismatch: ", paste(missing_cols, collapse = ", "))

  x <- X[, xnames, drop = FALSE]
  mtry_prop <- min(1, max(1 / ncol(x), rf$mtry / ncol(x)))
  nodesize_prop <- min(1, max(1 / nrow(x), rf$min.node.size / nrow(x)))

  set.seed(1304)
  pom <- Pomona::var.sel.perm(
    x          = as.data.frame(x),
    y          = tr$occ,
    no.perm    = altmann_permutations,
    p.t        = 0,
    ntree      = rf$num.trees,
    mtry.prop  = mtry_prop,
    nodesize.prop = nodesize_prop,
    no.threads = 1,
    method     = "ranger",
    type       = "regression",
    parametric = TRUE,
    importance  = "permutation"
  )
}
)

```

```

method_used <- "altmann_pomona"
pom_info <- as.data.frame(pom$info)
tibble(
  Variable = rownames(pom_info),
  PValue = as.numeric(pom_info[["pvalue"]])
),
error = function(e) {
  message(
    "Pomona failed for ", sp,
    "; trying ranger::importance_pvalues: ",
    conditionMessage(e)
  )

  tryCatch(
    {
      tr <- rename(model_sel$trainingData, occ = .outcome)
      pvals <- ranger::importance_pvalues(
        rf,
        method = "altmann",
        num.permutations = altmann_permutations,
        formula = occ ~ ., data = tr
      )
      method_used <- "altmann_ranger"
      tibble(Variable = names(pvals), PValue = as.numeric(pvals))
    },
    error = function(e2) {
      message(
        "importance_pvalues also failed; ",
        "using permutation-rank fallback: ",
        conditionMessage(e2)
      )
      method_used <- "permutation_rank"
      tibble(Variable = names(imp), PValue = NA_real_)
    }
  )
}
)

# Aggregate dummy-coded factors; select by p-value or importance rank
imp_tbl_agg <- imp_tbl %>%
  left_join(p_tbl, by = "Variable") %>%
  group_by(OriginalVar) %>%
  summarise(
    Importance = mean(Importance, na.rm = TRUE),
    PValue = min(PValue, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  rename(Variable = OriginalVar) %>%
  mutate(Species = sp, SelectionMethod = method_used)

if (any(!is.na(imp_tbl_agg$PValue))) {
  selected <- imp_tbl_agg %>%
    filter(Importance > 0, !is.na(PValue), PValue <= alpha_pvalue) %>%

```

```

    arrange(PValue, desc(Importance)) %>%
    pull(Variable)
  } else {
    rel_cut <- 0.20 * max(imp_tbl_agg$Importance, na.rm = TRUE)
    selected <- imp_tbl_agg %>%
      filter(is.finite(Importance), Importance >= rel_cut) %>%
      arrange(desc(Importance)) %>%
      pull(Variable)
  }

  # Enforce minimum number of retained predictors
  if (length(selected) < min_keep) {
    selected <- imp_tbl_agg %>%
      arrange(desc(Importance)) %>%
      slice_head(n = min(min_keep, nrow(imp_tbl_agg))) %>%
      pull(Variable)
  }

  selection_models[[sp]] <- model_sel
  selected_vars_abio[[sp]] <- unique(selected)
  selection_importance_tables[[sp]] <- imp_tbl_agg
}

# Export selection results
selection_importance_long <- bind_rows(selection_importance_tables)
selected_vars_long <- imap_dfr(
  selected_vars_abio,
  ~ tibble(Species = .y, Variable = .x)
)

write_csv(
  selection_importance_long,
  file.path(out_dir, "Selection_Importance_PValues.csv")
)
write_csv(
  selected_vars_long,
  file.path(out_dir, "Selected_Variables_Abiotic.csv")
)
saveRDS(
  selected_vars_abio,
  file.path(out_dir, "Selected_Variables_Abiotic.rds")
)
saveRDS(selection_models, file.path(out_dir, "Selection_Models_Abiotic.rds"))

message("Variable selection complete.")
print(selected_vars_long, n = Inf)

```

6. Final Abiotic Models — 1,000 Trees (Step 2)

```
# Helper: extract and aggregate variable importance
# (handles dummy-coded factors)
extract_importance <- function(model_list, model_group) {
  imap_dfr(model_list, function(model, sp) {
    imp <- ranger::importance(model$finalModel)

    orig_factor_map <- list()
    for (fv in factor_vars) {
      dummies <- grep(paste0("^", fv), names(imp), value = TRUE)
      for (dm in dummies) orig_factor_map[[dm]] <- fv
    }

    tibble(
      Variable = names(imp),
      Importance = as.numeric(imp),
      OriginalVar = vapply(names(imp), function(v) {
        if (v %in% names(orig_factor_map)) orig_factor_map[[v]] else v
      }, character(1))
    ) %>%
      group_by(OriginalVar) %>%
      summarise(
        Importance = mean(Importance, na.rm = TRUE),
        .groups = "drop"
      ) %>%
      rename(Variable = OriginalVar) %>%
      mutate(Species = sp, ModelGroup = model_group)
  })
}

# Helper: build spatial train control (falls back to standard repeatedcv)
build_control <- function(df_sp, coord_cols, block_size,
                          k_folds, cached = NULL) {
  if (!is.null(cached)) {
    return(cached)
  }

  sb <- tryCatch(
    {
      sf_df <- st_as_sf(df_sp, coords = coord_cols, crs = 2950, remove = FALSE)
      cv_spatial(
        x = sf_df, size = block_size, k = k_folds,
        selection = "random", iteration = 100, seed = 1304, progress = FALSE
      )
    },
    error = function(e) NULL
  )

  ctrl_args <- list(
    method      = "repeatedcv",
    number      = k_folds,
    repeats     = 3,
    savePredictions = "final",
  )
}
```

```

    allowParallel = TRUE,
    verboseIter   = FALSE
  )
  if (!is.null(sb)) {
    ctrl_args$index <- lapply(sb$fold_list, `[`, 1)
    ctrl_args$indexOut <- lapply(sb$fold_list, `[`, 2)
  }
  do.call(caret::trainControl, ctrl_args)
}

final_models_abio <- list()

for (sp in names(selected_vars_abio)) {
  message("[Final Abiotic] ", sp)

  preds <- intersect(selected_vars_abio[[sp]], names(trainData))
  if (length(preds) < 2) {
    message("Skipping ", sp, " - fewer than 2 selected predictors.")
    next
  }

  needed <- unique(c(coord_cols, preds, sp))
  df_sp <- trainData %>%
    select(any_of(needed)) %>%
    filter(complete.cases(.[, intersect(needed, names(.))]))

  if (nrow(df_sp) < 100) {
    message("Skipping ", sp, " - too few rows (", nrow(df_sp), ")")
    next
  }

  control <- build_control(df_sp, coord_cols, block_size, k_folds,
    cached = controls_by_species[[sp]]
  )

  p <- length(preds)
  tune_grid <- expand_grid(
    mtry = sort(unique(pmax(1, pmin(p, mtry_candidates)))),
    splitrule = "variance",
    min.node.size = c(5, 10)
  )

  model_df <- df_sp %>%
    select(all_of(c(preds, sp))) %>%
    rename(occ = all_of(sp)) %>%
    mutate(occ = as.numeric(occ)) %>%
    filter(complete.cases(.))

  set.seed(1305)
  final_models_abio[[sp]] <- caret::train(
    occ ~ .,
    data = model_df,
    method = "ranger",
    preProcess = pre_proc,

```

```

    metric      = "RMSE",
    trControl   = control,
    tuneGrid    = tune_grid,
    num.trees   = n_trees_final,
    importance   = "permutation"
  )
}

# CV performance summary
perf_abio <- imap_dfr(final_models_abio, function(model, sp) {
  best <- model$results %>%
    arrange(RMSE) %>%
    slice(1)
  tibble(
    Species      = sp,
    RMSE_CV      = best$RMSE,
    Rsquared_CV  = best$Rsquared,
    MAE_CV       = best$MAE,
    Best_mtry    = model$bestTune$mtry,
    Best_min_node_size = model$bestTune$min.node.size
  )
})

imp_abio <- extract_importance(final_models_abio, "abiotic")

write_csv(perf_abio, file.path(out_dir, "Model_Performance_Abiotic_CV.csv"))
write_csv(imp_abio, file.path(out_dir, "Variable_Importance_Abiotic.csv"))
saveRDS(final_models_abio, file.path(out_dir, "Final_Models_Abiotic.rds"))

print(perf_abio)

```

7. Test Set LOO VIS and Independent Test Set Construction

```

test_plain <- if (inherits(testData, "sf")) {
  st_drop_geometry(testData)
} else {
  as.data.frame(testData)
}
names(test_plain) <- make.unique(names(test_plain))

n_test <- test_plain %>%
  select(ends_with("_n")) %>%
  rename_with(~ sub("_n$", "", .x))
ba_test <- test_plain %>%
  select(ends_with("_ba")) %>%
  rename_with(~ sub("_ba$", "", .x))

stopifnot(nrow(n_test) == nrow(test_plain), nrow(ba_test) == nrow(test_plain))

```

```

species_loo_test <- intersect(
  species_list,
  intersect(colnames(n_test), colnames(ba_test))
)
if (length(species_loo_test) == 0)
  stop("No target species found for test LOO VIS computation.")

# Compute LOO VIS for test set
VIS_LOO_list_test <- setNames(
  vector("list", length(species_loo_test)),
  species_loo_test
)

for (target in species_loo_test) {
  message("Computing test LOO VIS for: ", target)
  VIS_LOO_list_test[[target]] <- calc_VIS_LOO(
    n_mat = n_test,
    ba_mat = ba_test,
    target = target,
    scale100 = TRUE,
    drop_target_col = TRUE
  )
}

# Build per-species independent test set
# (abiotic columns + LOO co-occurrence IV)
independent_test_set_list <- setNames(
  vector("list", length(species_list)),
  species_list
)

for (sp in species_list) {
  df_sp <- test_plain %>%
    select(any_of(c(abiotic_candidates, coord_cols, sp))) %>%
    cbind(VIS_LOO_list_test[[sp]])
  independent_test_set_list[[sp]] <- df_sp
}

# Sanity check: co-occurrence IVs should have non-zero variance
sds_sab <- sapply(
  VIS_LOO_list_test[["SAB"]][, setdiff(species_list, "SAB")],
  function(x) round(sd(x, na.rm = TRUE), 3)
)
message("LOO VIS SD check for SAB (co-occurring species): ")
print(sds_sab)

saveRDS(
  independent_test_set_list,
  file.path(out_dir, "independent_test_set_list.rds")
)
message(
  "Independent test set list built for ",
  length(independent_test_set_list), " species."
)

```

8. Final Abiotic + Co-occurrence Models — 1,000 Trees (Step 3)

The species-specific abiotic predictor set is kept unchanged. The leave-one-out-normalized IV of the six co-occurring prioritized species is appended directly as additional predictors. **No further variable selection is applied to the co-occurrence predictors.**

```
final_models_abio_bio <- list()

for (sp in names(selected_vars_abio)) {
  message("[Final Abiotic+Cooccur] ", sp)

  preds_abio <- intersect(selected_vars_abio[[sp]], names(trainData))

  # Append LOO IV of the other 6 focal species (no selection applied)
  loo_df <- as.data.frame(VIS_LOO_list[[sp]])
  cooccur_cols <- intersect(setdiff(species_list, sp), names(loo_df))

  if (length(cooccur_cols) > 0 && nrow(loo_df) == nrow(trainData)) {
    df_sp <- trainData %>%
      select(any_of(c(coord_cols, preds_abio, sp))) %>%
      bind_cols(loo_df[, cooccur_cols, drop = FALSE])
  } else {
    message(
      "LOO VIS unavailable or row-mismatched for ", sp,
      "; using raw IV as fallback."
    )
    cooccur_cols <- setdiff(species_list, sp)
    df_sp <- trainData %>%
      select(any_of(c(coord_cols, preds_abio, sp, cooccur_cols)))
  }

  predictors <- setdiff(names(df_sp), c(coord_cols, sp))
  if (length(predictors) < 2) next

  needed <- unique(c(coord_cols, predictors, sp))
  df_sp <- df_sp %>%
    filter(complete.cases(.[, intersect(needed, names(.))]))

  if (nrow(df_sp) < 100) next

  control <- build_control(df_sp, coord_cols, block_size, k_folds,
    cached = controls_by_species[[sp]]
  )

  p <- length(predictors)
  tune_grid <- expand_grid(
    mtry = sort(unique(pmax(1, pmin(p, mtry_candidates)))),
    splitrule = "variance",
    min.node.size = c(5, 10)
  )
}
```

```

model_df <- df_sp %>%
  select(all_of(c(predictors, sp))) %>%
  rename(occ = all_of(sp)) %>%
  mutate(occ = as.numeric(occ)) %>%
  filter(complete.cases(.))

set.seed(1306)
final_models_abio_bio[[sp]] <- caret::train(
  occ ~ .,
  data      = model_df,
  method    = "ranger",
  preProcess = pre_proc,
  metric     = "RMSE",
  trControl  = control,
  tuneGrid   = tune_grid,
  num.trees  = n_trees_final,
  importance = "permutation"
)
}

# CV performance summary
perf_abio_bio <- imap_dfr(final_models_abio_bio, function(model, sp) {
  best <- model$results %>%
    arrange(RMSE) %>%
    slice(1)
  tibble(
    Species      = sp,
    RMSE_CV      = best$RMSE,
    Rsquared_CV  = best$Rsquared,
    MAE_CV       = best$MAE,
    Best_mtry    = model$bestTune$mtry,
    Best_min_node_size = model$bestTune$min.node.size
  )
})

imp_abio_bio <- extract_importance(final_models_abio_bio, "abiotic_cooccur")

write_csv(
  perf_abio_bio,
  file.path(out_dir, "Model_Performance_Abiotic_Cooccur_CV.csv")
)
write_csv(
  imp_abio_bio,
  file.path(out_dir, "Variable_Importance_Abiotic_Cooccur.csv")
)
saveRDS(
  final_models_abio_bio,
  file.path(out_dir, "Final_Models_Abiotic_Cooccur.rds")
)

# Combined importance table across all models
importance_all <- bind_rows(imp_abio, imp_abio_bio)
write_csv(
  importance_all,

```

```

    file.path(out_dir, "Variable_Importance_All_Models.csv")
  )

print(perf_abio_bio)

```

9. Model Consolidation

```

all_rf_models <- list(
  SAB_abiotic      = final_models_abio[["SAB"]],
  SAB_abiotic_cooccur = final_models_abio_bio[["SAB"]],
  BOJ_abiotic      = final_models_abio[["BOJ"]],
  BOJ_abiotic_cooccur = final_models_abio_bio[["BOJ"]],
  BOP_abiotic      = final_models_abio[["BOP"]],
  BOP_abiotic_cooccur = final_models_abio_bio[["BOP"]],
  EPN_abiotic      = final_models_abio[["EPN"]],
  EPN_abiotic_cooccur = final_models_abio_bio[["EPN"]],
  EPB_abiotic      = final_models_abio[["EPB"]],
  EPB_abiotic_cooccur = final_models_abio_bio[["EPB"]],
  ERR_abiotic      = final_models_abio[["ERR"]],
  ERR_abiotic_cooccur = final_models_abio_bio[["ERR"]],
  ERS_abiotic      = final_models_abio[["ERS"]],
  ERS_abiotic_cooccur = final_models_abio_bio[["ERS"]]
)

```

10. Performance Evaluation — Training and Independent Test Set

10.1 Prediction Helpers

```

generate_predictions <- function(model, data) {
  if (inherits(data, "sf")) data <- st_drop_geometry(data)
  data <- as.data.frame(data)
  is_listcol <- vapply(data, is.list, logical(1))
  if (any(is_listcol)) data <- data[, !is_listcol, drop = FALSE]

  n <- nrow(data)
  out <- rep(NA_real_, n)

  if (inherits(model, "train")) {
    required_predictors <- setdiff(names(model$trainingData), ".outcome")
    if (length(required_predictors) == 0)
      stop("Could not infer predictors from model$trainingData.")

    for (pred in setdiff(required_predictors, names(data))) data[[pred]] <- NA
  }
}

```

```

X <- data[, required_predictors, drop = FALSE]

template <- model$trainingData
for (pred in intersect(required_predictors, names(template))) {
  if (is.factor(template[[pred]])) {
    X[[pred]] <- factor(
      as.character(X[[pred]]),
      levels = levels(template[[pred]])
    )
  } else if (is.numeric(template[[pred]]) && !is.numeric(X[[pred]])) {
    X[[pred]] <- suppressWarnings(as.numeric(as.character(X[[pred]])))
  }
}

ok <- complete.cases(X)
if (any(ok))
  out[ok] <- as.numeric(
    predict(model, newdata = X[ok, , drop = FALSE])
  )
return(out)
}

if (inherits(model, "randomForest")) {
  required_predictors <- model$xNames
  for (pred in setdiff(required_predictors, names(data))) data[[pred]] <- NA
  X <- data[, required_predictors, drop = FALSE]
  ok <- complete.cases(X)
  if (any(ok))
    out[ok] <- as.numeric(
      predict(model, newdata = X[ok, , drop = FALSE])
    )
  return(out)
}

stop("Model must be a caret::train or randomForest object.")
}

create_prediction_dataframe <- function(models,
                                         independent_test_set_list,
                                         species_map = NULL) {
  plot_data_list <- map(names(models), function(model_id) {
    model <- models[[model_id]]
    species_code <- str_extract(model_id, "[A-Z]+")

    if (is.na(species_code) ||
        !(species_code %in% names(independent_test_set_list))) {
      warning(
        "Skipping ", model_id,
        " - no matching test set for ", species_code
      )
    }
    return(NULL)
  })
}

```

```

test_df <- independent_test_set_list[[species_code]]

# Training predictions
if (inherits(model, "train")) {
  tr_df <- model$trainingData
  train_obs <- tr_df$outcome
  train_pred <- generate_predictions(model, tr_df)
} else {
  train_obs <- as.numeric(model$y)
  train_pred <- as.numeric(model$predicted)
}

train_results <- tibble(
  Observed = as.numeric(train_obs),
  Predicted = as.numeric(train_pred),
  DataType = "Training"
)

# Test predictions
test_results <- if (species_code %in% names(test_df)) {
  tibble(
    Observed = as.numeric(test_df[[species_code]]),
    Predicted = generate_predictions(model, test_df),
    DataType = "Test"
  )
} else {
  warning(
    "Observed column '", species_code,
    "' not found in test set. Skipping test."
  )
  NULL
}

species_label <- if (
  !is.null(species_map) && species_code %in% names(species_map)
) {
  unname(species_map[[species_code]])
} else {
  species_code
}

model_type <- if_else(
  str_detect(model_id, "_cooccur$"),
  "Abiotic + Co-occurrence",
  "Abiotic Only"
)

bind_rows(train_results, test_results) %>%
  mutate(
    model_id = model_id,
    Species = species_label,
    ModelType = model_type
  )
})

```

```

  bind_rows(plot_data_list)
}

```

10.2 Metrics Calculation

```

plot_data_full <- create_prediction_dataframe(
  all_rf_models, independent_test_set_list,
  species_map = species_map
)

write_csv(
  plot_data_full,
  file.path(out_dir, "Observed_vs_Predicted_all_models.csv")
)

metrics_data <- plot_data_full %>%
  group_by(Species, ModelType, DataType) %>%
  summarise(
    R2      = cor(Observed, Predicted, use = "complete.obs")^2,
    RMSE    = sqrt(mean((Observed - Predicted)^2, na.rm = TRUE)),
    Bias    = mean(Predicted - Observed, na.rm = TRUE),
    Pct_Bias = (
      sum(Predicted - Observed, na.rm = TRUE) * 100
    ) / sum(Observed, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    metric_label = sprintf(
      "R² = %.2f\nRMSE = %.2f\nBias = %.2f\nPct_Bias = %.2f%%",
      R2, RMSE, Bias, Pct_Bias
    )
  )

write_csv(
  metrics_data,
  file.path(out_dir, "Performance_Metrics_all_models.csv")
)
print(metrics_data, n = Inf)

```

11. Observed vs. Predicted Plots

```

lvl <- unname(species_map)

create_obs_pred_panel <- function(data, metrics, data_type, panel_label) {
  d <- data %>%

```

```

filter(
  ModelType %in% c("Abiotic Only", "Abiotic + Co-occurrence"),
  DataType == data_type
) %>%
mutate(
  Species = factor(Species, levels = lvl),
  ModelType = factor(
    ModelType,
    levels = c("Abiotic Only", "Abiotic + Co-occurrence")
  )
)

m <- metrics %>%
filter(
  ModelType %in% c("Abiotic Only", "Abiotic + Co-occurrence"),
  DataType == data_type
) %>%
mutate(
  Species = factor(Species, levels = lvl),
  ModelType = factor(
    ModelType,
    levels = c("Abiotic Only", "Abiotic + Co-occurrence")
  )
)

ggplot(d, aes(x = Observed, y = Predicted)) +
  geom_point(alpha = 0.2, size = 0.7, color = "#333333") +
  geom_abline(
    slope = 1, intercept = 0,
    color = "red", linetype = "dashed", linewidth = 0.8
  ) +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  stat_regline_equation(
    aes(label = ..eq.label..),
    label.x = 15, label.y = 5, size = 3, fontface = "bold", color = "black"
  ) +
  geom_text(
    data = m,
    aes(x = 5, y = 95, label = metric_label),
    inherit.aes = FALSE,
    hjust = 0, vjust = 1, size = 3, fontface = "bold", color = "black"
  ) +
  facet_grid(Species ~ ModelType) +
  coord_fixed(xlim = c(0, 100), ylim = c(0, 100), expand = FALSE) +
  labs(title = panel_label, x = "Observed IV (%)", y = "Predicted IV (%)") +
  theme_bw(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5, size = 13),
    strip.text = element_text(face = "bold", size = 10),
    strip.background = element_rect(fill = "grey90", color = NA),
    panel.grid.minor = element_blank(),
    legend.position = "none"
  )
}

```

```

p_train <- create_obs_pred_panel(
  plot_data_full, metrics_data,
  data_type = "Training",
  panel_label = "A. Model Performance on Training Data (Fit)"
)

p_test <- create_obs_pred_panel(
  plot_data_full, metrics_data,
  data_type = "Test",
  panel_label = "B. Model Performance on Independent Test Data (Generalization)"
)

combined_plot <- p_train + p_test

ggsave(
  file.path(out_dir, "Performance_Training_vs_Test.svg"),
  plot = combined_plot, width = 18, height = 12, units = "in", dpi = 96
)

print(p_train)
print(p_test)

```

12. Spatial Maps — Hexagon Maps

```

ensure_sf <- function(df, coords = c("longitude", "latitude"), crs = 2950) {
  if (inherits(df, "sf")) {
    return(df)
  }
  if (all(coords %in% names(df))) {
    return(st_as_sf(df, coords = coords, crs = crs))
  }
  st_as_sf(df)
}

all_data_raw <- bind_rows(trainData, testData)
map_data_all <- ensure_sf(all_data_raw, coords = coord_cols)

study_area_proj <- st_transform(
  st_make_valid(study_area),
  crs = st_crs(map_data_all)
)
study_area_hex_perim <- st_cast(
  st_union(study_area_proj), "MULTILINESTRING"
) |> st_as_sf()

hex_cellsize <- 5000
hex_grid <- st_make_grid(
  st_as_sfc(st_bbox(study_area_proj)),

```

```

    cellsize = hex_cellsize,
    square = FALSE
  )
  hex_grid_sf <- st_sf(
    hex_id = seq_along(hex_grid),
    geometry = hex_grid,
    crs = st_crs(map_data_all)
  )
  hex_grid_sf <- st_intersection(hex_grid_sf, study_area_proj)

  summarise_to_hex <- function(pts_sf, value_col, hex_sf) {
    pts_use <- pts_sf |>
      filter(!is.na(.data[[value_col]])) |>
      mutate(value_log = log1p(pmax(.data[[value_col]], 0))) |>
      select(value_log)

    if (nrow(pts_use) == 0) {
      return(mutate(hex_sf, value_hex = NA_real_))
    }

    pts_joined <- st_join(pts_use, hex_sf, left = FALSE)
    if (nrow(pts_joined) == 0) {
      return(mutate(hex_sf, value_hex = NA_real_))
    }

    hex_stats <- pts_joined |>
      st_drop_geometry() |>
      group_by(hex_id) |>
      summarise(value_hex = mean(value_log, na.rm = TRUE), .groups = "drop")

    hex_sf |> left_join(hex_stats, by = "hex_id")
  }

  hex_species_list <- species_list[
    species_list %in% names(all_data_raw) &
    paste0(species_list, "_abiotic") %in% names(all_rf_models)
  ]

  safe_predict <- function(model, newdata) {
    out <- rep(NA_real_, nrow(newdata))
    pred_vars <- intersect(predictors(model), names(newdata))
    ok <- complete.cases(newdata[, pred_vars, drop = FALSE])
    if (any(ok)) {
      out[ok] <- tryCatch(
        as.numeric(predict(model, newdata = newdata[ok, , drop = FALSE])),
        error = function(e) {
          warning(e)
          rep(NA_real_, sum(ok))
        }
      )
    }
    out
  }

```

```

panel_theme <- theme_minimal(base_size = 11) +
  theme(
    axis.text = element_blank(),
    axis.ticks = element_blank(),
    axis.title = element_blank(),
    panel.grid = element_blank(),
    panel.border = element_blank(),
    plot.title = element_text(size = 11, face = "bold", hjust = 0.5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 9),
    plot.margin = ggplot2::margin(t = 2, r = 2, b = 2, l = 2, unit = "pt")
  )

make_hex_map <- function(hex_data, title_text, fill_limits) {
  ggplot() +
    geom_sf(data = hex_data, aes(fill = value_hex), colour = NA) +
    geom_sf(
      data = study_area_hex_perim,
      colour = "black", linewidth = 0.45, fill = NA
    ) +
    scale_fill_viridis_c(
      option = "inferno",
      limits = fill_limits,
      oob = scales::squish,
      name = "Log-transformed\nimportance value\n(log[IV + 1])",
      na.value = "white"
    ) +
    labs(title = title_text) +
    coord_sf(expand = FALSE, datum = NA) +
    panel_theme
}

for (sp in hex_species_list) {
  cat(glue("Generating 4-panel hex map (observed + predicted) for {sp}...\n"))

  model_abio <- all_rf_models[[paste0(sp, "_abiotic")]]
  model_cooc <- all_rf_models[[paste0(sp, "_abiotic_cooccur")]]

  pred_abio_vals <- safe_predict(model_abio, all_data_raw)
  pred_cooc_vals <- if (!is.null(model_cooc)) {
    safe_predict(model_cooc, all_data_raw)
  } else {
    rep(NA_real_, nrow(all_data_raw))
  }

  pts_sf <- map_data_all |>
    mutate(
      obs_iv = .data[[sp]],
      pred_abio = pred_abio_vals,
      pred_cooc = pred_cooc_vals
    )

  hex_obs <- summarise_to_hex(pts_sf, "obs_iv", hex_grid_sf)
  hex_abio <- summarise_to_hex(pts_sf, "pred_abio", hex_grid_sf)
}

```

```

hex_cooc <- summarise_to_hex(pts_sf, "pred_cooc", hex_grid_sf)

hex_diff <- hex_grid_sf |>
  select(hex_id) |>
  left_join(
    hex_cooc |>
      st_drop_geometry() |>
      select(hex_id, value_hex) |>
      rename(value_cooc = value_hex),
    by = "hex_id"
  ) |>
  left_join(
    hex_abio |>
      st_drop_geometry() |>
      select(hex_id, value_hex) |>
      rename(value_abio = value_hex),
    by = "hex_id"
  ) |>
  mutate(value_hex = value_cooc - value_abio)

fill_limits <- range(
  c(hex_obs$value_hex, hex_abio$value_hex, hex_cooc$value_hex),
  na.rm = TRUE
)
if (!all(is.finite(fill_limits))) fill_limits <- c(0, 1)

diff_lim <- max(abs(hex_diff$value_hex), na.rm = TRUE)
if (!is.finite(diff_lim)) diff_lim <- 1

p1 <- make_hex_map(hex_obs, "(a) Observed", fill_limits)
p2 <- make_hex_map(hex_abio, "(b) Predicted (Abiotic)", fill_limits)
p3 <- make_hex_map(
  hex_cooc,
  "(c) Predicted (Abiotic + Co-occurrence)",
  fill_limits
)

p4 <- ggplot() +
  geom_sf(data = hex_diff, aes(fill = value_hex), colour = NA) +
  geom_sf(
    data = study_area_hex_perim,
    colour = "black", linewidth = 0.45, fill = NA
  ) +
  scale_fill_gradient2(
    low = "#4575b4",
    mid = "white",
    high = "#8c510a",
    midpoint = 0,
    limits = c(-diff_lim, diff_lim),
    oob = scales::squish,
    name = paste0(
      "Difference\n(Abiotic + Co-occurrence",
      "\nminus Abiotic)\nlog(IV + 1)"
    ),
  ),

```

```

    na.value = "white"
  ) +
  labs(title = "(d) Difference") +
  coord_sf(expand = FALSE, datum = NA) +
  panel_theme

sp_name_key <- paste0(sp, "_n")
pretty_name <- if (sp_name_key %in% names(qc_species_names)) {
  qc_species_names[[sp_name_key]]
} else {
  sp
}

p_final <- (p1 + p2) / (p3 + p4) +
  plot_layout(guides = "collect") +
  plot_annotation(
    title = glue("{pretty_name}"),
    subtitle = glue(
      "Training + Test | Hexagons ({hex_cellsize / 1000} km cellsize)"
    ),
    theme = theme(
      plot.title = element_text(size = 13, face = "bold", hjust = 0.5),
      plot.subtitle = element_text(size = 10.5, hjust = 0.5)
    )
  ) &
  theme(
    legend.position = "bottom",
    legend.direction = "horizontal",
    legend.box = "vertical",
    legend.key.width = unit(1.5, "cm")
  )

ggsave(
  filename = file.path(
    out_dir, "maps",
    glue("map_hex_obs_pred_4panels_{sp}.png")
  ),
  plot = p_final,
  width = 10.5, height = 8.2, dpi = 300, bg = "white"
)
}

cat("Hex maps (observed + predicted, train + test) complete.\n")

```

13. Study Area Map with Inset

```

plots_all <- read_csv(
  "database/undisturbed_plots_sf_env_clim_4th_inv_study_area_PET.csv",
  show_col_types = FALSE
)

```

```

) %>%
  st_as_sf(coords = c("longitude", "latitude"), crs = 2950)

# Mark training vs test by coordinate matching
train_key <- paste(
  round(trainData[[coord_cols[1]]], 6),
  round(trainData[[coord_cols[2]]], 6)
)
plots_key <- paste(
  round(st_coordinates(plots_all)[, 1], 6),
  round(st_coordinates(plots_all)[, 2], 6)
)

plots_all <- plots_all %>%
  mutate(
    Data_Partition = if_else(plots_key %in% train_key, "Training", "Test")
  )

world <- ne_countries(scale = "medium", returnclass = "sf")

main_map <- ggplot() +
  geom_sf(data = study_area, fill = "white", color = "black", linewidth = 0.8) +
  geom_sf(
    data = plots_all %>% filter(Data_Partition == "Training"),
    aes(fill = MAT),
    shape = 21, color = "grey35", size = 2, alpha = 0.85, stroke = 0.01
  ) +
  geom_sf(
    data = plots_all %>% filter(Data_Partition == "Test"),
    aes(fill = MAT),
    shape = 21, color = "black", size = 2, alpha = 0.9, stroke = 0.5
  ) +
  scale_fill_gradientn(
    name = "MAT (°C)",
    colours = colorRampPalette(c("#67a9cf", "#f2f0e6", "#ef8a62"))(100),
    limits = c(0, 4)
  ) +
  annotation_scale(location = "br", width_hint = 0.3, style = "ticks") +
  annotation_north_arrow(
    location = "t1",
    which_north = "true",
    pad_x = unit(0.2, "in"),
    pad_y = unit(0.3, "in"),
    style = north_arrow_fancy_orienteering()
  ) +
  labs(
    title = "Study Area and Plot Locations",
    subtitle = "EPU 1415 NC, Québec, Canada",
    x = "Longitude", y = "Latitude"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", size = 14, hjust = 0.5),
    plot.subtitle = element_text(size = 11, hjust = 0.5),

```

```

    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.8),
    panel.grid.major = element_line(color = "gray90", linewidth = 0.3),
    legend.position = "right"
  )

inset_map <- ggplot(world) +
  geom_sf(fill = "lightgray", color = "white", linewidth = 0.3) +
  geom_sf(
    data      = study_area %>% st_transform(crs = 4326),
    fill      = "red",
    color     = "darkred",
    alpha     = 0.7,
    linewidth = 0.3
  ) +
  coord_sf(xlim = c(-100, -50), ylim = c(20, 60), expand = FALSE) +
  labs(title = "Location in\nEastern North America") +
  theme_minimal(base_size = 12) +
  theme(
    plot.title   = element_text(face = "bold", size = 10, hjust = 0.5),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.4),
    axis.text    = element_blank(),
    axis.title   = element_blank(),
    axis.ticks   = element_blank()
  )

bbox <- st_bbox(study_area)
final_map <- main_map +
  annotation_custom(
    grob = ggplotGrob(inset_map),
    xmin = bbox["xmax"] - (bbox["xmax"] - bbox["xmin"]) * 0.40,
    xmax = Inf,
    ymin = bbox["ymax"] - (bbox["ymax"] - bbox["ymin"]) * 0.30,
    ymax = Inf
  )

print(final_map)
ggsave(
  file.path(out_dir, "Study_Area_Map_with_Inset.svg"),
  plot = final_map, width = 10, height = 8, dpi = 96, bg = "white"
)

cat("Study area map saved.\n")

```

Export PDF and HTML Reports

```

library(rmarkdown)

render(
  paste0(

```

```
    "G:/Thesis/2ndChapter/script/",  
    "000_CzarneckiLiz_RForest_Phytosociological_Proxies_IV_prediction.Rmd"  
  ),  
  output_format = "pdf_document",  
  output_file   = file.path(  
    "G:/Thesis/2ndChapter/results/RF_models_20260626_111002",  
    "modeling_report.pdf"  
  )  
)
```